Antmicro

Topwrap

2024-03-28

DOCUMENTATION

1 Introduction 1
1.1 Wrappers . . . 1
1.2 Top module . . . 1

2 Getting started 3
2.1 Installation . . . 3
2.2 Example usage . . . 3

3 Description files 7
3.1 Design Description . . . 7
3.2 IP description files . . . 9
3.3 Interface Description files . . . 12

4 Interconnect generation 13
4.1 Format . . . 13
4.2 Interconnect params . . . 14
4.3 Limitations . . . 14

5 Command-line Interface 15
5.1 Building design . . . 15
5.2 Connect Topwrap to Pipeline Manager . . . 15
5.3 Generating IP core description YAMLs . . . 16

6 Kenning Pipeline Manager 17
6.1 Run Topwrap with Pipeline Manager . . . 17
6.2 Pipeline Manager features . . . 20

7 Wrapper Port 22

8 IPWrapper class 23

9 IPConnect class 24

10 ElaboratableWrapper class 25

11 Helpers 26

12 FuseSoC 27
12.1 Core file template . . . 27
12.2 Using the FuseSocBuilder . . . 27

13 Setup **28**

14 Code style **29**
14.1 Lint with nox 29
14.2 Lint with pre-commit 29
14.3 Tools 30

15 Tests **31**
15.1 Test execution 31

16 Examples **32**

17 Future enhancements **33**
17.1 Creating hierarchical block designs 33
17.2 Bus management 33
17.3 Improve the process of recreating a design from a YAML file 33
17.4 Support for parsing SystemVerilog sources 34
17.5 Provide a way to parse HDL sources from the Pipeline Manager level 34
17.6 Ability to produce top-level wrappers in VHDL 34

CHAPTER

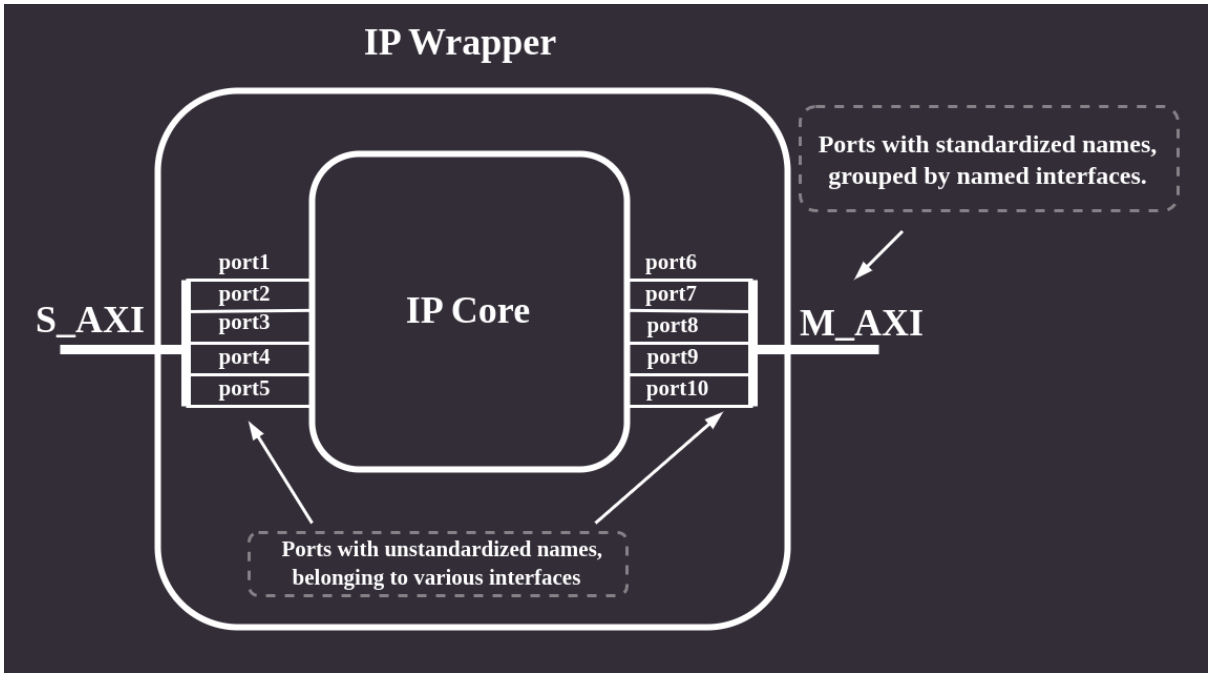
ONE

INTRODUCTION

Topwrap is a tool for generating HDL wrappers for simple blocks as well as digital systems and can be used for creating complex SoCs from single blocks contained in wrappers.

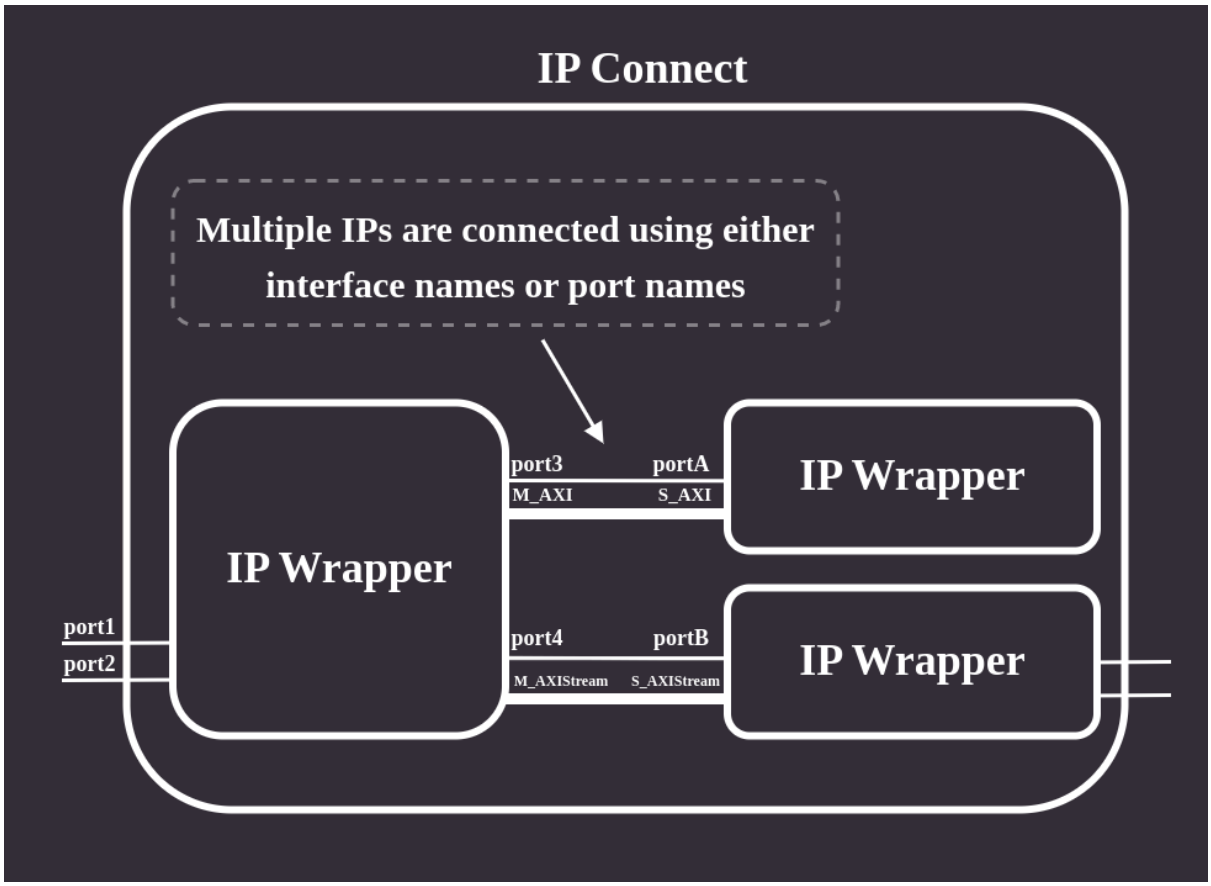
1.1 Wrappers

These wrappers standardize names of ports that belong to an interfaces (e.g. UART, AXI etc.)



1.2 Top module

Top modules connect IPs and/or Wrappers by either ports` names or interfaces` names to ease connecting multi-wire interfaces without the need to connect each wire separately.


IP Connect
Multiple IPs are connected using either interface names or port names
IP Wrapper
port1
port2
port3
portA
M_AXI
S_AXI
IP Wrapper
port4
portB
M_AXIStream
S_AXIStream
IP Wrapper

CHAPTER

TWO

GETTING STARTED

2.1 Installation

1. Install required system packages:

```
apt install -y git g++ make python3 python3-pip antlr4 libantlr4-runtime-dev␣
↪yosys
```

2. It is highly recommended to run the following steps in a Python virtual environment (e.g. venv).
3. Install the Topwrap package:

```
pip3 install .
```

Note: To use `topwrap parse` command you also need to install optional dependencies:

```
pip3 install ".[topwrap-parse]"
```

2.2 Example usage

This section demonstrates the basic usage of Topwrap to generate IP wrappers and a top module.

1. Create `IPWrapper` objects for every IP Core you want to use:

```yaml
ips:
  dma:
    file: ips/DMATop.yaml
    module: DMATop
  disp:
    file: ips/axi_dispctrl_v1_0.yaml
    module: axi_dispctrl_v1_0
```

`file` and `module` are mandatory fields providing the IP description and the name of the HDL module. You can optionally set parameter values for IP cores:

```yaml
axi_interconnect0:
  file: axi_interconnect.yaml
  module: axi_interconnect
  parameters:
    S_COUNT: 1
    M_COUNT: 3
    ADDR_WIDTH: 32
    DATA_WIDTH: 32
    ID_WIDTH: 12
    M_BASE_ADDR: 0x43c2000043c1000043c00000
    M_ADDR_WIDTH:
        value: 0x100000001000000010
        width: 96
```

2. Connect desired ports of the IP cores:

```yaml
ports:
  dma:
    clock: [some_other_ip, clk_out]
    reset: [reset_core, reset0]
  axi_interconnect0:
    clk: [some_other_ip, clk_out]
```

3. Connect desired interfaces of the IP cores:

```yaml
interfaces:
  dma:
    s_axi: [axi_interconnect0, m_axi_0]
  axi_interconnect0:
    m_axi: [some_other_ip, 's_axi']
```

4. Specify external ports or interfaces of the top module and connect them with chosen IP cores' ports or interfaces:

```yaml
ports:
  dma:
    io_irq_readerDone: readerDone
    io_irq_writerDone: writerDone

...

external:
  ports:
    out:
    - readerDone
    - writerDone
  interfaces:
    ...
```

5. Create a Core file template for FuseSoC, or use a default one bundled with Topwrap.

You may want to modify the file to add dependencies, source files, or change the target board.

The file should be named `core.yaml.j2`. You can find an example template in `examples/hdmi` directory of the project. If you don't create any template a default template bundled with Topwrap will be used (stored in `templates` directory).

6. Place any additional source files in a directory, which we will be calling `sources` here.
7. Run Topwrap:

```
python -m topwrap build --design project.yml --sources sources
```

2.2.1 Example PWM design

There's an example setup in `examples/pwm`.

In order to generate the top module, run:

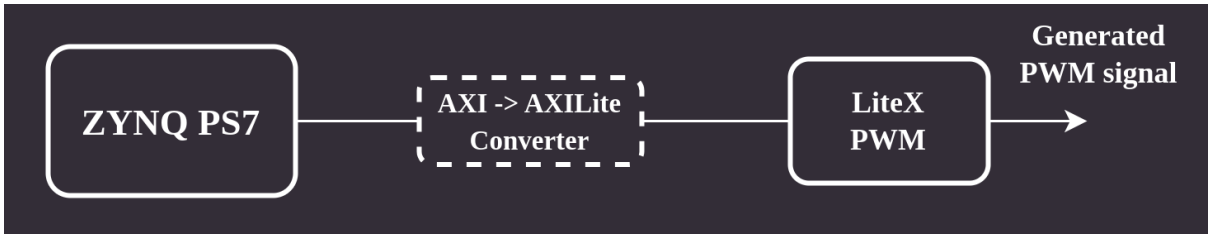
```
make generate
```

In order to generate bitstream, add Vivado to your path and run:

```
make
```

The FPGA design utilizes an AXI-mapped PWM IP Core. You can access its registers starting from address `0x4000000` (that's the base address of `AXI_GP0` on ZYNQ). Each IP Core used in the project is declared in `ips` section in `project.yml` file. `ports` section allows to connect individual ports of IP Cores, and `interfaces` is used analogously for connecting whole interfaces. Finally, you can specify which ports will be used as external I/O in `external` section.

To connect the I/O signals to specific FPGA pins, you need proper mappings in a constraints file. See `zynq.xdc` used in the setup and modify it accordingly.



2.2.2 Example HDMI design

There's an example setup stored in `examples/hdmi`.

You can generate bitstream for desired target:

- Snickerdoodle Black:

```
make snickerdoodle
```

- Zynq Video Board:

```
make zvb
```

If you wish to generate HDL sources without running Vivado, you can use:

```
make generate
```

You can find more information in README of the example setup.

CHAPTER

THREE

DESCRIPTION FILES

3.1 Design Description

To create a complete, fully synthesizable design, a proper design file is needed. It's used to specify interconnects, IP cores, set their parameters' values, describe hierarchies for the project, connect the IPs and hierarchies, and pick external ports (those which will be connected to physical I/O).

You can see example design files in `examples` directory. The structure is as below:

```
ips:
  # specify relations between IPs names in the design yaml
  # and IP cores description yamls and modules
  {ip_instance_name}:
    file: {path_to_yaml_file_of_the_ip}
    module: {name_of_the_HDL_module}
  ...

design:
  name: {design_name} # optional name of the toplevel
  hierarchies: # specify hierarchies designs
    {hierarchy_name_1}:
      design:
        parameters:
          ...
        ports: # ports connections internal to this hierarchy
          ...
        interfaces: # interfaces connections internal to this hierarchy
          ...
        {nested_hierarchy_name}:
          ...
      external:
        # external ports and/or interfaces of this hierarchy; these can be
        # referenced in the upper-level `ports`, `interfaces` or `external`␣
↪section
        ...
    {hierarchy_name_2}:
      ...

  parameters: # specify IPs parameter values to be overridden
```

(continues on next page)

(continued from previous page)

```
    {ip_instance_name}:
      {param_name} : {param_value}
      ...

  ports:
    # specify incoming ports connections of an IP named `ip1_name`
    {ip1_name}:
      {port1_name} : [{ip2_name}, {port2_name}]
      ...
    # specify incoming ports connections of a hierarchy named `hier_name`
    {hier_name}:
      {port1_name} : [{ip_name}, {port2_name}]
      ...
    # specify external ports connections
    {ip_instance_name}:
      {port_name} : ext_port_name
    ...

  interfaces:
    # specify incoming interfaces connections of `ip1_name` IP
    {ip1_name}:
      {iface1_name} : [{ip2_name}, {iface2_name}]
      ...
    # specify incoming interfaces connections of `hier_name` hierarchy
    {hier_name}:
      {iface1_name} : [{ip_name}, {iface2_name}]
      ...
    # specify external interfaces connections
    {ip_instance_name}:
      {iface_name} : ext_iface_name
    ...

  interconnects:
    # see "Interconnect generation" page for a detailed description of the format
    ...

external: # specify names of external ports and interfaces of the top module
  ports:
    out:
      - {ext_port_name}
    inout:
      - [{ip_name/hierarchy_name, port_name}]
  interfaces:
    in:
      - {ext_iface_name}
```

In order for an IP to be present in the generated design, it must be specified in `ports` or `interfaces` sections as a key. This means that even if it has no incoming connections from any other IP or hierarchy, the `ports` or `interfaces` sections must contain `ip_name: {}` entry.

`inout` ports are handled differently than `in` and `out` ports. When any IP has an inout port or when a hierarchy has an inout port specified in its `external.ports.inout` section, it must be included in `external.ports.inout` section of the parent design by specifying the name of the IP/hierarchy and port name that contains it. Name of the external port will be identical to the one in the IP core. In case of duplicate names a suffix `$n` is added (where `n` is a natural number) to the name of the second and subsequent duplicate names. `inout` ports cannot be connected to each other.

The design description yaml format allows creating hierarchical designs. In order to create a hierarchy, it suffices to add its name as a key in the `design` section and describe the hierarchy design "recursively" by using the same keys and values (`ports`, `parameters` etc.) as in the top-level design (see above). Hierarchies can be nested recursively, which means that you can create a hierarchy inside another one.

Note that IPs and hierarchies names cannot be duplicated on the same hierarchy level. For example, the `design` section cannot contain two identical keys, but it's correct to have `ip_name` key in this section and `ip_name` in the `design` section of some hierarchy.

3.2 IP description files

Every IP wrapped by Topwrap needs a description file in YAML format.

The ports of an IP should be placed in global `signals` node, followed by the direction of `in`, `out` or `inout`. Here's an example description of ports of Clock Crossing IP:

```yaml
# file: clock_crossing.yaml
signals:
    in:
        - clkA
        - A
        - clkB
    out:
        - B
```

The previous example is enough to make use of any IP. However, in order to benefit from connecting whole interfaces at once, ports must belong to a named interface like in this example:

```yaml
#file: axis_width_converter.yaml
s_axis:
    interface: AXIStream
    mode: slave
    signals:
        in:
            TDATA: [s_axis_tdata, 63, 0]
            TKEEP: [s_axis_tkeep, 7, 0]
            TVALID: s_axis_tvalid
            TLAST: s_axis_tlast
            TID: [s_axis_tid, 7, 0]
            TDEST: [s_axis_tdest, 7, 0]
            TUSER: s_axis_tuser
        out:
```

(continues on next page)

(continued from previous page)

```
            TREADY: s_axis_tready

m_axis:
    interface: AXIStream
    mode: master
    signals:
        in:
            TREADY: m_axis_tready
        out:
            TDATA: [m_axis_tdata, 31, 0]
            TKEEP: [m_axis_tkeep, 3, 0]
            TVALID: m_axis_tvalid
            TLAST: m_axis_tlast
            TID: [m_axis_tid, 7, 0]
            TDEST: [m_axis_tdest, 7, 0]
            TUSER: m_axis_tuser
signals: # These ports don't belong to any interface
    in:
        - clk
        - rst
```

Names `s_axis` and `m_axis` will be used to group the selected ports. Each signal in an interface has a name which must match with the signal it's supposed to be connected to, for example `TDATA: port_name` will be connected to `TDATA: other_port_name`.

Note that you don't have to write IP core description yamls by hand. You can use Topwrap's `parse` command (see *Generating IP core description YAMLs*) in order to generate yamls from HDL source files and then adjust the yaml to your needs.

3.2.1 Port widths

The width of every port defaults to `1`. You can specify the width using this notation:

```
s_axis:
    interface: AXIStream
    mode: slave
    signals:
        in:
            TDATA: [s_axis_tdata, 63, 0] # 64 bits
            ...
            TVALID: s_axis_tvalid # defaults to 1 bit

signals:
    in:
        - [gpio_io_i, 31, 0] # 32 bits
```

3.2.2 Parameterization

Port widths don't have to be hardcoded - you can use parameters to describe an IP core in a generic way. Values specified in IP core yamls can be overridden in a design description file (see *Design Description*).

```
parameters:
    DATA_WIDTH: 8
    KEEP_WIDTH: (DATA_WIDTH+7)/8
    ID_WIDTH: 8
    DEST_WIDTH: 8
    USER_WIDTH: 1

s_axis:
    interface: AXI4Stream
    mode: slave
    signals:
        in:
            TDATA: [s_axis_tdata, DATA_WIDTH-1, 0]
            TKEEP: [s_axis_tkeep, KEEP_WIDTH-1, 0]
            ...
            TID: [s_axis_tid, ID_WIDTH-1, 0]
            TDEST: [s_axis_tdest, DEST_WIDTH-1, 0]
            TUSER: [s_axis_tuser, USER_WIDTH-1, 0]
```

Parameters values can be integers or math expressions, which are evaluated using `numexpr.evaluate()`.

3.2.3 Port slicing

You can also slice a port, to use some bits of the port as a signal that belongs to an interface. The example below means:

`Port m_axi_bid of the IP core is 36 bits wide. Use bits 23..12 as the BID signal of AXI master named m_axi_1`

```
m_axi_1:
    interface: AXI
    mode: master
    signals:
        in:
            BID: [m_axi_bid, 35, 0, 23, 12]
```

3.3 Interface Description files

Topwrap can use predefined interfaces which are described in YAML-formatted files. The interfaces you use don't have to be predefined, but it's possible to perform checks on whether all the mandatory signals are connected, when you use an interface definition.

You can see an example file below:

```yaml
name: AXI4Stream
port_prefix: AXIS
signals:
    required:
        - TVALID
        - TDATA
        - TLAST
        - TREADY
    optional:
        - TID
        - TDEST
        - TKEEP
        - TSTRB
        - TUSER
```

The name of an interface has to be unique. We also specify a prefix which will be used as a shortened identifier.

CHAPTER

FOUR

INTERCONNECT GENERATION

Generating interconnects is an experimental feature of topwrap. With a specification of which interfaces are masters or slaves and their address ranges, topwrap is able to automatically generate an interconnect conforming to this description. Currently supported interconnect types are:

- Wishbone round-robin

4.1 Format

The format for describing interconnects is specified below. `interconnects` key must be a direct descendant of the `design` key in the design description.

```
interconnects:
  {interconnect1_name}:
    # specify clock and reset to drive the interconnect with
    clock: [{ip_name, clk_port_name}]
    reset: [{ip_name, rst_port_name}]
    # alternatively you can specify a connection to an external interface:
    # clock: ext_clk_port_name
    # reset: ext_rst_port_name

    # specify interconnect type
    type: {interconnect_type}

    # specify interconnect parameters - interconnect-type-dependent (see
↪"Interconnect params" section):
    params:
      {param_name1}: param_value1
      ...

    # specify masters and their interfaces connected to the bus
    masters:
    {master1_name}:
      - {master1_iface1_name}
      ...
    ...

    # specify slaves, their interfaces connected to the bus and their bus␣
```

(continues on next page)

(continued from previous page)

```
↪parameters
    slaves:
    {slave1_name}:
      {slave1_interface1_name}:
      # requests in address range [address, address+size) will be routed to this␣
↪interface
      address: {start_address}
      size: {range_size}
      ...
    ...
```

4.2 Interconnect params

Different interconnect types may provide different configuration options. This section lists parameter names for available interconnects for use in the `params` section of interconnect specification.

4.2.1 Wishbone round-robin

Corresponds to `type: wishbone_roundrobin`

- `addr_width` - bit width of the address line (addresses access `data_width`-sized chunks)
- `data_width` - bit width of the data line
- `granularity` - access granularity - smallest unit of data transfer that the interconnect is capable of transferring. Must be one of: 8, 16, 32, 64
- `features` - optional, list of optional wishbone signals, can contain: `err`, `rty`, `stall`, `lock`, `cti`, `bte`

4.3 Limitations

Known limitations currently are:

- only word-sized addressing is supported (in other words - consecutive addresses access word-sized chunks of data)
- crossing clock domains is not supported
- down-converting (initiating multiple transactions on a narrow bus per one transaction on a wider bus) is not supported
- up-converting is not supported

CHAPTER

FIVE

COMMAND-LINE INTERFACE

5.1 Building design

To run Topwrap, use:

```
python -m topwrap build --design project.yml
```

Where `project.yml` should be your file with description of the top module.

You can specify a directory to be scanned for additional sources:

```
python -m topwrap build --sources src --design project.yml
```

To implement the design for a specific FPGA chip, provide the part name:

```
python -m topwrap build --sources src --design project.yml --part 'xc7z020clg400-3
↪'
```

5.2 Connect Topwrap to Pipeline Manager

If you want to use Pipeline Manager as a UI for creating block design, you need to:

1. Build and run Pipeline Manager server application.

```
python -m topwrap kpm_build_server
python -m topwrap kpm_run_server
```

2. Run Topwrap's client application, that will connect to a running Pipeline Manager server app.

```
python -m topwrap kpm_client [-h ip_addr] [-p port] FILES
```

Topwrap will then try to connect to the server running on `ip_addr:port` and send a specification generated from `FILES`, which should be IP core description yamls.

If `-h` or `-p` options are not specified, ip address `127.0.0.1` and port `9000` will be chosen by default.

5.3 Generating IP core description YAMLs

You can also use Topwrap to generate ip core description yamls from HDL sources, that can be later used in your `project.yml`:

```
python -m topwrap parse HDL_FILES
```

In HDL source files, ports that belong to the same interface (e.g. wishbone or AXI), have often a common prefix, which corresponds to the interface name. If such naming convention is followed in the HDL sources, Topwrap can also divide ports into user-specified interfaces, or automatically deduce interfaces names when generating yaml file:

```
python -m topwrap parse --iface wishbone --iface s_axi HDL_FILES

python -m topwrap parse --iface-deduce HDL_FILES
```

To get help, use:

```
python -m topwrap [build|kpm_client|parse] --help
```

CHAPTER

SIX

KENNING PIPELINE MANAGER

Topwrap can make use of Kenning Pipeline Manager to visualize the process of creating block design.

6.1 Run Topwrap with Pipeline Manager

1. Build and run Pipeline Manager server

   In order to start creating block design in Pipeline Manager, you need to first build and run a server application - here is a brief instruction on how to achieve this (the process of building and installation of Pipeline Manager is described in detail in its documentation):

   ```
   python -m topwrap kpm_build_server
   python -m topwrap kpm_run_server
   ```

   After executing the above-mentioned commands, the Pipeline Manager server is waiting for an external application (i.e. Topwrap) to connect on `127.0.0.1:9000` and you can connect to the web GUI frontend in your browser on `http://127.0.0.1:5000`.

2. Establish connection with Topwrap

   Once the Pipeline Manager server is running, you can now launch Topwrap's client application in order to connect to the server. You need to specify:

   - IP address (`127.0.0.1` is default)
   - listening port (`9000` is default)
   - yamls describing IP cores, that will be used in the block design

   An example command, that runs Topwrap's client, may look like this:

   ```
   python -m topwrap kpm_client -h 127.0.0.1 -p 9000 \
       topwrap/ips/axi/axi_axil_adapter.yaml \
       examples/pwm/ipcores/{litex_pwm.yml,ps7.yaml}
   ```

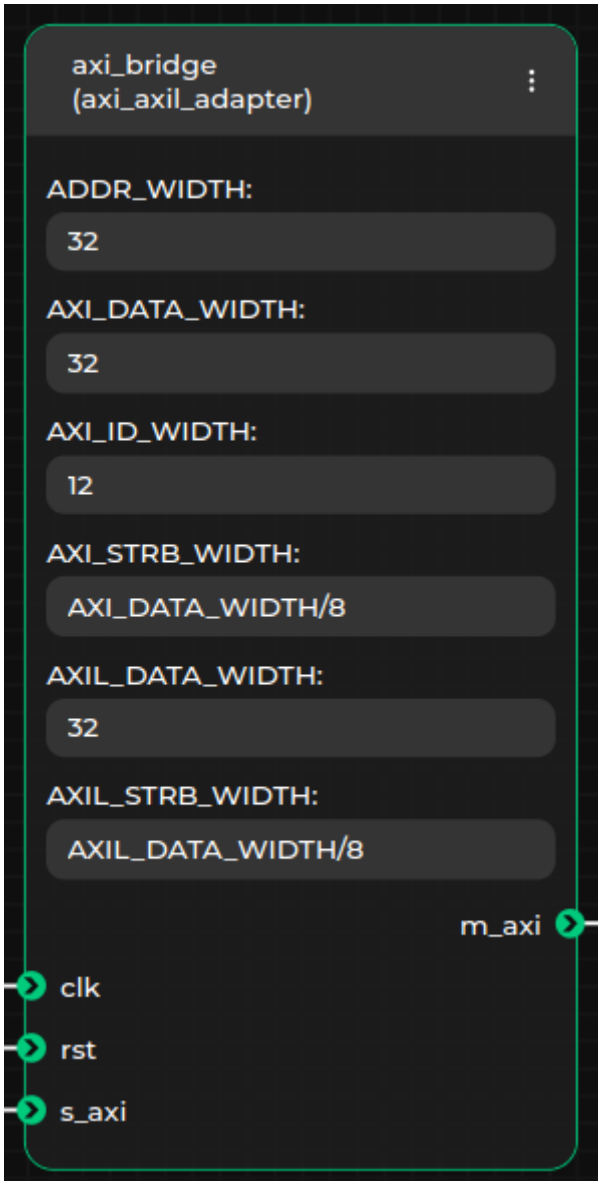
3. Create block design in Pipeline Manager

   Upon successful connection to a Pipeline Manager server, Topwrap will generate and send to the server a specification describing the structure of previously selected IP cores. After that, you are free to create a custom block design by means of:

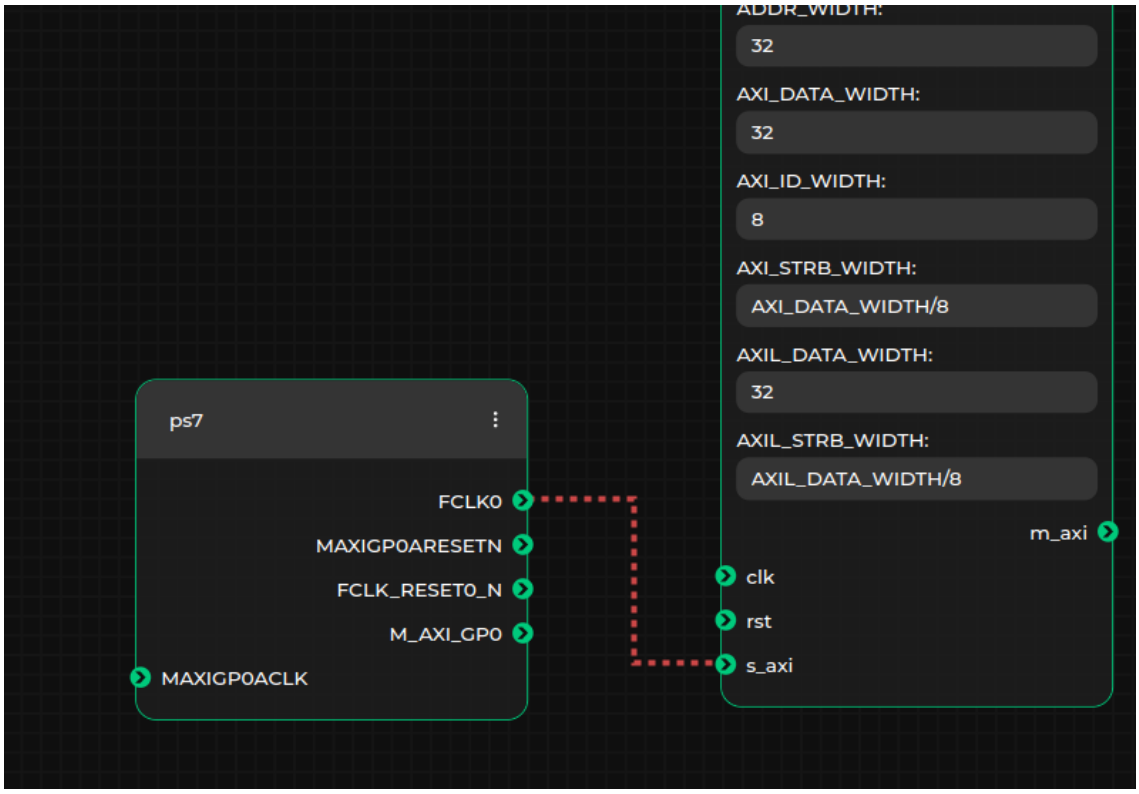
   - adding IP core instances to the block design. Each Pipeline Manager's node has `delete` and `rename` options, which make it possible to remove the selected node and

change its name respectively. This means that you can create multiple instances of the same IP core.

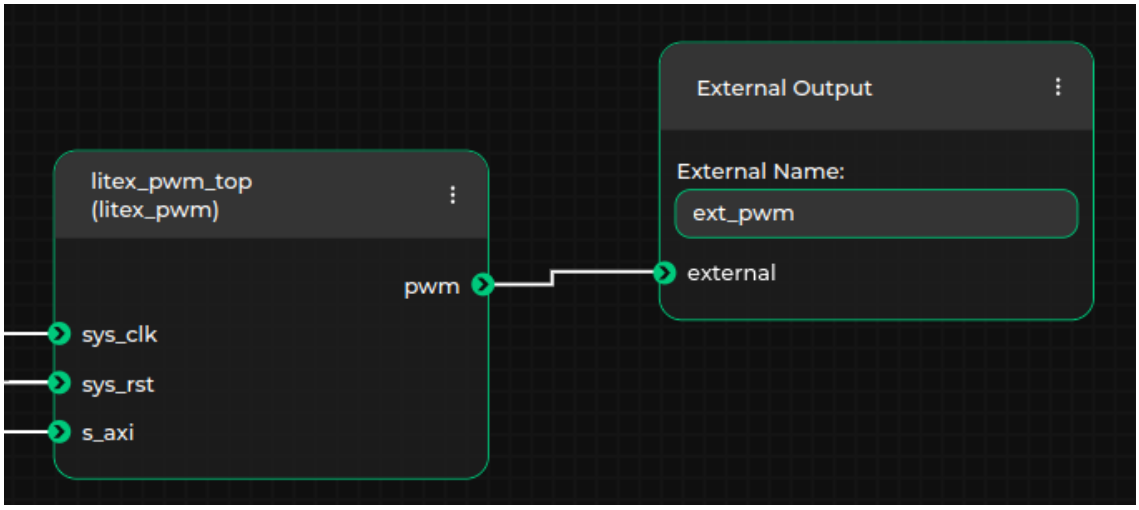
- adjusting IP cores' parameters values. Each node may have input boxes in which you can enter parameters' values (default parameter values are added while adding an IP core to the block design):



- connecting IP cores' ports and interfaces. Only connections between ports or interfaces of matching types are allowed. This is automatically checked by Pipeline Manager, as the types of nodes' ports and interfaces are contained in the loaded specification, so Pipeline Manager will prevent you from connecting non-matching interfaces (e.g. *AXI4* with *AXI4Lite* or a port with an interface). A green line will be displayed if a connection is possible to create, or a red line elsewhere:

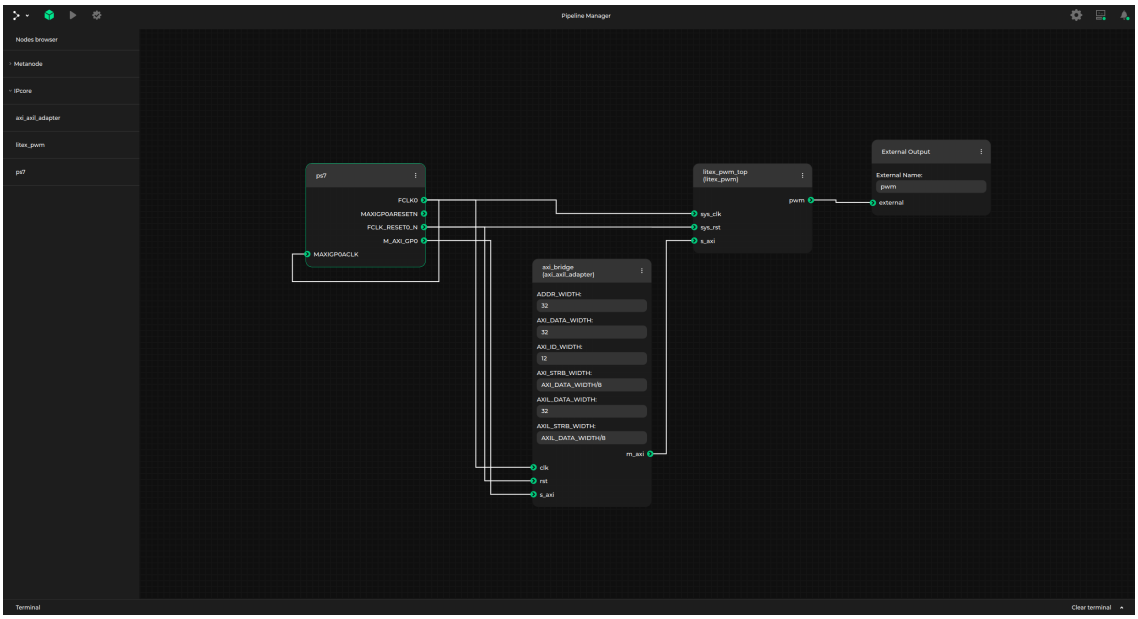


- specifying external ports or interfaces in the top module. This can be done by adding `External Input`, `External Output` or `External Inout` metanodes and creating connections between them and chosen ports or interfaces. Note that you should adjust the name of the external port or interface in a textbox inside selected metanode. In the example below, output port `pwm` of `litex_pwm_top` IP core will be made external in the generated top module and the external port name will be set to `ext_pwm`:



Note, that you don't always have to create a new block design by hand - you can use a *design import* feature to load an existing block design from a description in Topwrap's yaml format.

An example block design in Pipeline Manager for the PWM project may look like this:



6.2 Pipeline Manager features

While creating a custom block design, you can make use of the following Pipeline Manager's features:

- export (save) design to a file
- import (load) design from a file
- validate design
- build design

6.2.1 Export design to yaml description file

Created block design can be saved to a *design description file* in yaml format, using Pipeline Manager's `Save file` option. The design description file name will be generated automatically based on current timestamp.

6.2.2 Import design from yaml description file

Topwrap also supports conversion in the opposite way - block design in Pipeline Manager can be generated from a yaml design description file using `Load file` feature.

6.2.3 Design validation

Pipeline Manager is capable of performing some basic checks at runtime such as interface type checking while creating a connection. However you can also run more complex tests by using Pipeline Manager's `Validate` option. Topwrap will then respond with a validity confirmation or error messages. The rules you need to follow in order to keep your block design valid are:

- multiple IP cores with the same name are not allowed (except from external metanodes).
- parameters values can be integers of different bases (e.g. `0x28`, `40` or `0b101000`) or arithmetic expressions, that are later evaluated using numexpr.evaluate() function (e.g. `(AXI_DATA_WIDTH+1)/4` is a valid parameter value assuming that a parameter named `AXI_DATA_WIDTH` exists in the same IP core). You can also write a parameter value in a Verilog format (e.g. `8'b00011111` or `8'h1F`) - in such case it will be interpreted as a fixed-width bit vector.
- a single port or interface cannot be external and connected to another IP core at the same time.
- connections between two external metanodes are not allowed.
- all the created external output or inout ports must have unique names. Only multiple input ports of IP cores can be driven be the same external signal.

Topwrap can also generate warnings if:

- some ports or interfaces remain unconnected.
- multiple ports are connected to an `External Input` metanode with an empty `External Name` property. If a block design validation returns a warning, it means that the block design can be successfully built, but it is recommended to follow the suggestion and resolve a particular issue.

6.2.4 Building design

Once the design has been created and tested for validity, you can build design using `Run` feature. If the design does not contain any errors, this will result in creating a top module, similarly when using Topwrap's `topwrap build` command.

CHAPTER

SEVEN

WRAPPER PORT

Class `WrapperPort` is an extension to Amaranth's `Signal`. It wraps a port, adding a new name and optionally slicing the signal. It adds these attributes:

```
WrapperPort.internal_name     # name of the port in internal source to be wrapped
WrapperPort.direction         # DIR_FANIN, DIR_FANOUT or DIR_NONE
WrapperPort.interface_name    # name of the group of ports (interface)
WrapperPort.bounds            # range of bits that belong to the port
                              # and range which is sliced from the port
```

See *Port slicing* to know more about `bounds`.

This is used in `IPWrapper` class implementation and there should be no need to use `WrapperPort` individually.

CHAPTER

EIGHT

IPWRAPPER CLASS

`IPWrapper` is the main class used to instantiate an IP Core and generate HDL wrapper. It's used to standardize names of ports that belong to interfaces to ease connecting multiple IPs.

CHAPTER

NINE

IPCONNECT CLASS

Topwrap uses `IPConnect` to encapsulate IP Cores in a module, and connect their wires and interfaces.

After the object is created, IPs can be added using `add_ip` method:

```
# create a wrapper for an IP
dma := IPWrapper('DMATop.yaml', ip_name='DMATop', instance_name='DMATop0'),

ipc = IPConnect()
ip.add_ip(dma)

# add other IPs
```

IPs are connected using `make_connections` function. Ports and interfaces can be set as external input/output/inout with `make_external_ports_interfaces` and retrieved with `get_ports`. For a wider example of using the `IPConnect` see *Getting Started*

CHAPTER

TEN

ELABORATABLEWRAPPER CLASS

`ElaboratableWrapper` encapsulates an Amaranth's Elaboratable and exposes an interface compatible with other wrappers which allows making connections with them. Supplied elaboratable must contain a `signature` property and a conforming interface as specified by Amaranth docs. Ports' directionality, their names and widths are inferred from it.

CHAPTER

ELEVEN

HELPERS

CHAPTER

TWELVE

FUSESOC

Toprwap uses FuseSoC to automate project generation and build process.

A `Core` file is needed in order to generate a project. The file can be generated from a template using `FuseSocBuilder` class. The `Core` file contains information about source files and synthesis tools.

12.1 Core file template

A template for the core file is bundled with Topwrap (`templates/core.yaml.j2`). You may need to edit the file to change the backend tool, set additional `Hooks` and change the FPGA part name or other parameters. By default, `FuseSocBuilder` searches for the template file in the directory you work in, so you should first copy the template into the project's location.

12.2 Using the FuseSocBuilder

Here's an example of how to generate a simple project:

```
from topwrap.fuse_helper import FuseSocBuilder
fuse = FuseSocBuilder()

# add source of the IPs used in the project
fuse.add_source('DMATop.v', 'verilogSource')

# add source of the top file
fuse.add_source('top.v', 'verilogSource')

# specify the names of the Core file and the directory where sources are stored
# generate the project
fuse.build('build/top.core', 'sources')
```

Now you can run FuseSoC to generate bitstream and program FPGA:

```
fusesoc --cores-root build run project_1
```

This requires having the suitable backend tool in your `PATH` (Vivado, for example).

CHAPTER

THIRTEEN

SETUP

It is recommended for developers to keep code style and frequently run tests. In order to setup the developer's environment install optional dependency groups `topwrap-parse`, `tests` and `lint` specified in `pyproject.toml` which include `nox` and `pre-commit`:

```
python -m venv venv
source venv/bin/activate
pip install ".[topwrap-parse,tests,lint]"
```

CHAPTER

FOURTEEN

CODE STYLE

Automatic formatting and linting of the code can be performed with either `nox` or `pre-commit`.

14.1 Lint with nox

After successful setup, `nox` sessions can be executed to perform lint checks:

```
nox -s isort black flake8
```

Note: To reuse current virtual environment and avoid long installation time use `-R` option:

```
nox -R -s isort black flake8
```

Note: pre-commit can also be run from nox:

```
nox -s pre_commit
```

14.2 Lint with pre-commit

Alternatively, you can use pre-commit to perform the same job. `Pre-commit` hooks need to be installed:

```
pre-commit install
```

Now, each use of `git commit` in the shell will trigger actions defined in the `.pre-commit-config.yaml` file. Pre-commit can be easily disabled with a similar command:

```
pre-commit uninstall
```

If you wish to run `pre-commit` asynchronously, then use:

```
pre-commit run --all-files
```

Note: `pre-commit` by default also runs `nox` with `isort`,`flake8` and `black` sessions:

14.3 Tools

Tools used in project for maintaining code style:

- `Nox` is a tool, which simplifies management of Python testing. Visit nox website
- `Pre-commit` is a framework for managing and maintaining multi-language pre-commit hooks. Visit pre-commit website
- `Black` is a code formatter. Visit black website
- `Flake8` is a tool capable of linting, styling fixes and complexity analysis. Visit flake8 website
- `Isort` is a Python utility to sort imports alphabetically. Visit isort website

CHAPTER

FIFTEEN

TESTS

Topwrap functionality is validated with tests, leveraging the `pytest` library.

15.1 Test execution

Tests are located in the `tests` directory. All tests can be run with `nox` by specifying the tests session:

```
nox -s tests
```

Note: To reuse existing virtual environment and avoid long installation time use `-R` option:

```
nox -R -s tests
```

Note: To force a specific Python version and avoid running tests for all listed versions, use `-p VERSION` option:

```
nox -p 3.10 -s tests
```

Tests can also be launched without `nox` by executing:

```
pytest
```

CHAPTER

SIXTEEN

EXAMPLES

Note: Basic usage of examples is explained in the *Getting started* section.

Examples provided with this project should cover from very simple designs to complex fully synthesizable cores. They should be sorted by increasing complexity and number of used features, e.g:

- 101: minimal base design
- 102: introduce user to parameters
- 103: introduce user to slicing
- 104: introduce user to interfaces
- 105: etc.

Developers are encouraged to create/add new examples in the same spirit. Simple examples are used to teach how to use this tool and demonstrate its features. Real-world use cases are also welcome to prove that the implementation is mature enough to handle practical designs.

Note: The goal of this project is not to create a library of cores and/or their functional validation. This need is fulfilled by projects like FuseSoC, SVUnit, etc.

CHAPTER

SEVENTEEN

FUTURE ENHANCEMENTS

17.1 Creating hierarchical block designs

Currently, Topwrap supports only creating "flat" designs; one can add IP cores and create connections between them, but there is no way of organizing specific parts of the design into hierarchies. Having such a facility implemented would make it easier to create complex designs.

Moreover, this feature is closely related to subgraphs, which will be supported shortly in Pipeline Manager. In the future, we'd also like to provide a possibility to create hierarchical designs in the Pipeline Manager, which would be a huge improvement in terms of organizing complex designs.

17.2 Bus management

Another goal we'd like to achieve is to enable users to create full-featured designs with processors by providing proper support for bus management. This should include features such as:

- ability to specify the address of a peripheral device on the bus
- support for the most popular buses or the ones that we use (AXI, wishbone, Tile-link)

This will require writing or creating bus arbiters (round-robin, crossbar) and providing a mechanism for connecting master(s) and slave(s) together. As a result, the user should be able to create complex SoC with Topwrap.

17.3 Improve the process of recreating a design from a YAML file

One of the main features that are supported by Topwrap and Pipeline Manager is exporting and importing user-created design to or from a design description YAML. However, during these conversions, information about the positions of user-added nodes is not preserved. This is cumbersome in the case of complicated designs since the imported nodes cannot be placed in the optimal positions.

Therefore, one of our objectives is to provide a convenient way of creating and restoring user-created designs in Pipeline Manager, so that the user doesn't have to worry about node positions when importing a design to Pipeline Manager.

17.4 Support for parsing SystemVerilog sources

Information about IP cores is stored in *IP core description YAMLs*. These files can be generated automatically from HDL source files - currently Verilog and VHDL are supported. Our goal is to provide the possibility of generating such YAMLs from SystemVerilog too.

17.5 Provide a way to parse HDL sources from the Pipeline Manager level

Another issue related to HDL parsing is that the user has to manually parse HDL sources to obtain the IP core description YAMLs. Then the files need to be provided as command-line parameters when launching the Topwrap Pipeline Manager client application. Therefore, we aim to provide a way of parsing HDL files and including them in the editor directly from the Pipeline Manager level.

17.6 Ability to produce top-level wrappers in VHDL

Topwrap now uses Amaranth to generate top-level design in Verilog. We would also like to add the ability to produce such designs in VHDL.